Contributing a Plugin to the QAH Git Marketplace

A practical guide for external users: from a plugin folder to a merged pull request

Acxel Orozco
Quantum Analysis Helper

November 2026 – Git Plugin Marketplace 1.1.1

Abstract

This document explains how any GitHub user (a "custom user") can ask to upload a plugin to the public QAH Git Marketplace. Uploads are **pull requests**: a contributor publishes the plugin to their own fork, opens a pull request, the validation workflow checks the catalog and archives, and the repository owner reviews and merges. Deletion is owner-only. No contributor ever receives the owner's token.

Contents

1 Purpose and audience **3**

2 Before you start **3**
2.1 What you need . . . 3
2.2 Public source warning . . . 3
2.3 Plugins must be distributable by you . . . 3

3 Plugin folder requirements **4**

4 Create your own GitHub token **4**
4.1 Option 1: GitHub CLI (easiest) . . . 4
4.2 Option 2: classic personal access token (recommended for PRs) . . . 5
4.3 Option 3: fine-grained token . . . 5
4.4 Token security . . . 5

5 Configure the plugin **5**

6 Option A: automatic pull request (recommended) **6**
6.1 GUI . . . 6
6.2 CLI . . . 6
6.3 What happens behind the scenes . . . 6

7 Option B: manual fork and pull request **7**

8 Validation: the `validate` check **7**
8.1 Common validation failures . . . 8

9 Review, merge and publication **8**

10 Updating an existing plugin **8**

11 Deletion and ownership **8**

12 Troubleshooting **9**

A Catalog structure **9**

B PR checklist **10**

C Command reference **10**

1 Purpose and audience

The QAH Git Marketplace is a public Git repository that stores a catalog (`index.json`) and plugin archives (`.qplugin.zip`) for Quantum Analysis Helper (QAH). It is read by the *Git Plugin Marketplace* plugin.

Upstream
ADOBApps/qah-git-marketplace, branch `master`. Public read; protected branch.

Fork
your own copy, `https://github.com/<you>/qah-git-marketplace`.

Pull request (PR)
your proposed change, reviewed and merged by the owner.

Token
a personal access token created in *your* GitHub account.

CI
the `validate` GitHub Actions check that must pass before merging.

There are two supported workflows:

1. **Automatic** (recommended): the plugin or CLI forks the repository, commits to a branch and opens the PR for you.
2. **Manual**: you fork and publish to your fork yourself, then open the PR on GitHub.

Both produce the same result. The owner merges; the plugin becomes available to everyone after users refresh the marketplace.

2 Before you start

2.1 What you need

- A GitHub account.
- A completed QAH plugin folder (at minimum `manifest.json` and `__init__.py`).
- A *new* version number for this submission.
- Your own GitHub token (Section 4).
- Git Plugin Marketplace 1.1.1 or newer, or the standalone `git_marketplace_cli.py`.

2.2 Public source warning

Everything committed to the public marketplace is publicly downloadable. A QAH Python plugin archive normally contains its `.py` source files. Do not submit private, paid, NDA-covered or secret code to the public repository. Use a separate private staging repository for such plugins.

2.3 Plugins must be distributable by you

Submit only plugins you own or are allowed to redistribute. The owner may reject a submission that appears to violate licensing or ownership.

3 Plugin folder requirements

The publisher accepts the classic QAH plugin layout:

```
my_plugin/
  manifest.json      required
  __init__.py        required (Plugin class)
  README.md          optional, copied to the catalog
  icon.svg           optional
  ...                any other runtime files
```

Listing 1: Minimal plugin folder

Required manifest fields:

Field	Example	Notes
`id`	`my_plugin`	letters, digits, `.`, `_` or `-`
`name`	My Plugin	display name
`version`	`1.2.0`	new version; semantic versioning recommended
`author`	Your Name	non-empty
`description`	Short text	non-empty
`category`	`tools`	`analysis`, `development`, `tools`, `visualization`

Example:

```
{
  "name": "My Plugin",
  "id": "my_plugin",
  "version": "1.2.0",
  "author": "Your Name",
  "description": "What the plugin does",
  "type": "python",
  "category": "tools",
  "main": "__init__.py",
  "dependencies": [],
  "license": { "type": "free", "max_activations": 3 }
}
```

Listing 2: manifest.json

The publisher builds the archive with `manifest.json` at the root. Do not ship a wrapper folder, `.git`, `__pycache__`, `dist`, `target`, virtual environments or old `.qplugin.zip` files; they are excluded automatically.

4 Create your own GitHub token

A token is a personal credential. It is created in *your* GitHub account; the owner cannot create one for you and must never share theirs.

4.1 Option 1: GitHub CLI (easiest)

```
gh auth login
```

Listing 3: Terminal

After that the plugin can read the token with `gh auth token`; no manual copying is needed as long as `gh` is on `PATH`.

4.2 Option 2: classic personal access token (recommended for PRs)

1. Sign in to your GitHub account.
2. Open https://github.com/settings/tokens/new.
3. Note: `QAH Git Marketplace contributor`.
4. Expiration: 30, 60 or 90 days.
5. Scope: `public_repo`.
6. Generate the token and copy it once.

`public_repo` can fork the public marketplace, write to your fork and open pull requests. This is the simplest option for the automatic submission.

4.3 Option 3: fine-grained token

1. Open https://github.com/settings/personal-access-tokens/new.
2. Resource owner: your account (or the organisation that owns your fork).
3. Repository access:
   - upstream `ADOBApps/qah-git-marketplace`: **Contents: Read**, **Pull requests: Read and write**;
   - your fork `<you>/qah-git-marketplace`: **Contents: Read and write**.
4. Metadata: Read (mandatory).
5. Generate and copy the token.

Fork caveat. A fine-grained token can only grant access to repositories that exist when the token is created. If you have not forked yet, either:

- create the fork manually first, then create the token; or
- use a classic `public_repo` token; or
- grant the token access to "All repositories" (not recommended).

4.4 Token security

- Never send your token to the owner or paste it into a public issue.
- Never commit it. The plugin stores it in `~/.qah/git_marketplace.json` (mode 0600 on Linux/macOS; user-profile ACL on Windows).
- Use the minimum scope and a short expiration; revoke it when done.
- A read-only token can browse and install but cannot publish or delete.

5 Configure the plugin

Open **QAH** → **Plugins** → **Git Plugin Marketplace** → **Settings** and fill in:

Field	Value
Owner	`ADOBApps`
Repository	`qah-git-marketplace`
Branch	`master`
Token	your token (Section 4)

Click **Test connection**. A read-only token reports `write: no`; that is expected for contributors. The public catalog remains readable without any token.

The same settings can be written from the CLI:

```
py -3 git_marketplace_cli.py config --owner ADOBApps \
    --repo-name qah-git-marketplace --branch master
py -3 git_marketplace_cli.py doctor
```

Listing 4: CLI configuration

6 Option A: automatic pull request (recommended)

6.1 GUI

1. Open **Git Plugin Marketplace**.
2. Click **Publish Plugin...**.
3. Select your plugin folder. The dialog reads `manifest.json`.
4. Review/edit name, author, category, description and changelog.
5. Tick **Submit as a pull request (fork + PR)**. It is pre-selected automatically when your token is read-only.
6. Click **Submit pull request**.

The plugin creates or reuses your fork, commits the catalog entry and archive to a temporary branch, opens the PR and shows the URL.

6.2 CLI

```
py -3 git_marketplace_cli.py submit "C:\path\to\my_plugin" \
    --changelog "Initial release: what it does"
```

Listing 5: Automatic submission

Useful flags: `-version 1.2.0`, `-draft`, `-title "..."`, `-homepage "..."`. Without `-version` the manifest version is used.

6.3 What happens behind the scenes

1. The folder is validated and archived as a `.qplugin.zip`.
2. The current upstream `index.json` is read and a new entry is added.
3. Your GitHub login is identified from the token.
4. Your fork is created (if missing) and synced with upstream.
5. A branch `marketplace/<id>-<version>-<timestamp>` is created from the upstream head.

6. `index.json`, `plugins/<id>/manifest.json`, optional README and the ZIP are committed to that branch in **one commit**.
7. A pull request is opened against `ADOBApps:master`.
8. If the PR cannot be opened, the temporary branch is deleted.

For the repository owner, the same action uses a branch in the upstream repository instead of a fork (owner same-repository PR).

7 Option B: manual fork and pull request

Use this if you prefer Git over the automatic action.

1. On GitHub, open ADOBApps/qah-git-marketplace and click **Fork**.
2. Configure the plugin to point at your fork: Owner `<you>`, Repository `qah-git-marketplace`, Branch `master`, your token.
3. Publish to your fork:

```
py -3 git_marketplace_cli.py --repo <you>/qah-git-marketplace \
    --branch master publish "C:\path\to\my_plugin" \
    --changelog "What changed"
```

Listing 6: Publish to your fork

4. On GitHub, open a pull request from `<you>:master` to `ADOBApps:master`.
5. Switch your plugin settings back to `ADOBApps` if you want to read the upstream catalog again.

If the upstream moved while you were working, sync your fork (GitHub's **Sync fork** button or the automatic action) and publish again. The PR updates automatically.

8 Validation: the `validate` check

Every PR runs `.github/workflows/validate-marketplace.yml`. The job checks:

- `index.json` is valid JSON with schema 1;
- plugin ids and versions are unique and safe;
- every local ZIP exists and its size and SHA256 match the catalog;
- the ZIP contains `manifest.json` at its root;
- the manifest `id` and `version` match the catalog entry;
- required manifest fields are non-empty;
- there are no absolute paths, `..` entries or symlinks;
- the catalog `version` equals the highest non-yanked version.

A green check means the data is consistent. It does not mean the plugin has been reviewed; the owner still reviews and merges.

8.1 Common validation failures

Message	Fix
`sha256 mismatch`	Publish again; do not edit the ZIP by hand.
`size ... != catalog ...`	Rebuild and republish the same version.
`manifest.json is not at the archive root`	The publisher does this for you; do not repack with a wrapper folder.
`manifest id ... != ...`	The folder manifest and the catalog entry disagree; republish from the correct folder.
`manifest version ... != ...`	Use the same version in the manifest and `-version`.
`missing ...`	The catalog references a file that is not in the PR.
`catalog version ... != latest ...`	Publish a higher version or fix `index.json`.

9 Review, merge and publication

- `master` is protected: PR required, one approval, Code Owner review, required `validate` check, no force pushes, no deletions.
- Only the owner (`@ADOBApps`) can merge. Contributors cannot push to `master`.
- The owner may request changes; update your fork/PR and republish.
- After the merge, the catalog is live immediately. Users click **Refresh** in Git Plugin Marketplace and see the new version.

Please keep one plugin/version per PR and do not modify other plugins in the same change.

10 Updating an existing plugin

Publish a new version with the same `id`. Older versions remain in the catalog, so users can still install them.

```
py -3 git_marketplace_cli.py submit "C:\path\to\my_plugin" \
    --version 1.3.0 --changelog "Fix ..., add ..."
```

Listing 7: Publishing an update

Never reuse a version number for different bytes. The catalog replaced the entry if the version already exists, but this breaks reproducibility for users.

11 Deletion and ownership

- Only the owner has a write token and can delete via the plugin (**Delete from marketplace**) or by merging a deletion PR.
- A contributor may open a PR that removes an entry, but it cannot be merged without the owner.
- Deletion removes the entry from the branch tip. Git history still contains the old objects; this is not a secure erase.

12 Troubleshooting

Symptom	Cause and fix
`401` / bad credentials	Token expired or mistyped. Create a new one and update Settings.
`403` on publish	Token lacks Contents write on the target repo, or the organisation requires SSO authorisation.
`403` on submit	Token cannot fork or open PRs. Use `public_repo`, or grant Pull requests write on upstream.
`404`	Private repository without access, or the fork is not ready yet. Wait and retry; check the owner/repo spelling.
`422` on branch	Branch already exists (rare timestamp collision) or the branch moved. Retry.
`422` on PR	A PR for this branch may already exist, or the base branch changed. Check GitHub.
Fork creation timeout	GitHub is slow. Retry after a minute; the fork may already exist.
`writable: false`	Your token is read-only; use **Submit as a pull request**, not direct publish.
CI failed	Read the job log; usually a SHA/size mismatch, missing file, or version not highest. Republish the same version.
Plugin missing after merge	Click **Refresh**; restart QAH if the menu does not update.

If the problem persists, open an issue at ADOBApps/qah-git-marketplace with the command, the exact error text and the plugin id/version. Never paste your token.

A Catalog structure

```
{
  "schema": 1,
  "name": "QAH Git Marketplace",
  "updated": "2026-11-06T22:31:00Z",
  "plugins": [
    {
      "id": "my_plugin",
      "name": "My Plugin",
      "version": "1.2.0",
      "author": "Your Name",
      "description": "What it does",
      "category": "tools",
      "dependencies": [],
      "versions": [
        {
          "version": "1.2.0",
          "path": "plugins/my_plugin/1.2.0/my_plugin-1.2.0.qplugin.zip",
          "sha256": "...64 hex characters...",
          "size": 12345,
          "changelog": "Initial release",
          "published": "2026-11-06T22:31:00Z",
          "yanked": false
        }
      ]
```

```
    }
  ]
}
```

Listing 8: index.json (abridged)

B PR checklist

- ☐ I own or may redistribute this plugin.
- ☐ The folder has `manifest.json` and `__init__.py`.
- ☐ The version is new and matches the manifest.
- ☐ `validate` passes.
- ☐ No tokens, credentials or private data are included.
- ☐ I understand the public repository exposes the archive contents.

C Command reference

```
# settings / diagnosis
git_marketplace_cli.py config --owner ADOBApps \
    --repo-name qah-git-marketplace --branch master --token <TOKEN>
git_marketplace_cli.py doctor

# automatic pull request
git_marketplace_cli.py submit <folder> --changelog "..." [--draft]

# manual fork workflow
git_marketplace_cli.py --repo <you>/qah-git-marketplace --branch master \
    publish <folder> --changelog "..."

# local validation / build
git_marketplace_cli.py validate <folder>
git_marketplace_cli.py build <folder> --out dist

# after the owner merges (public read, no token)
git_marketplace_cli.py list --refresh
git_marketplace_cli.py install <plugin-id>
```

Listing 9: Git Plugin Marketplace CLI